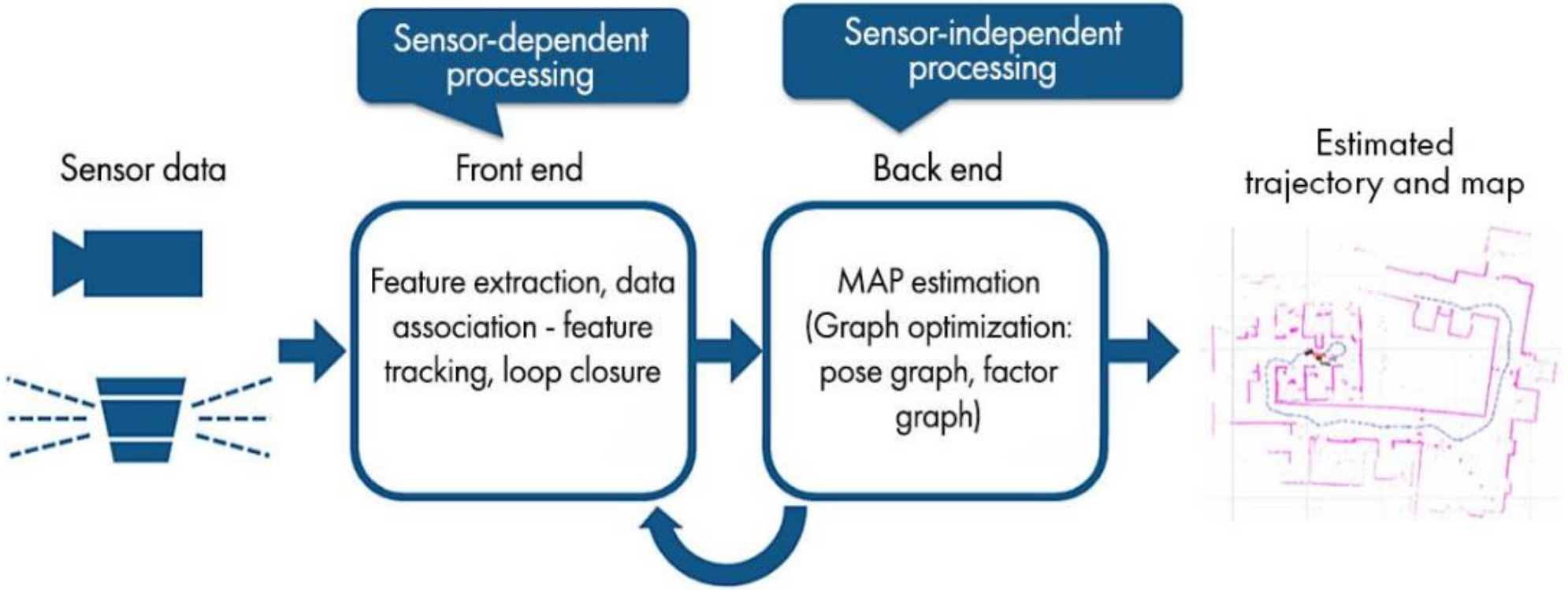
SLAM

SLAM

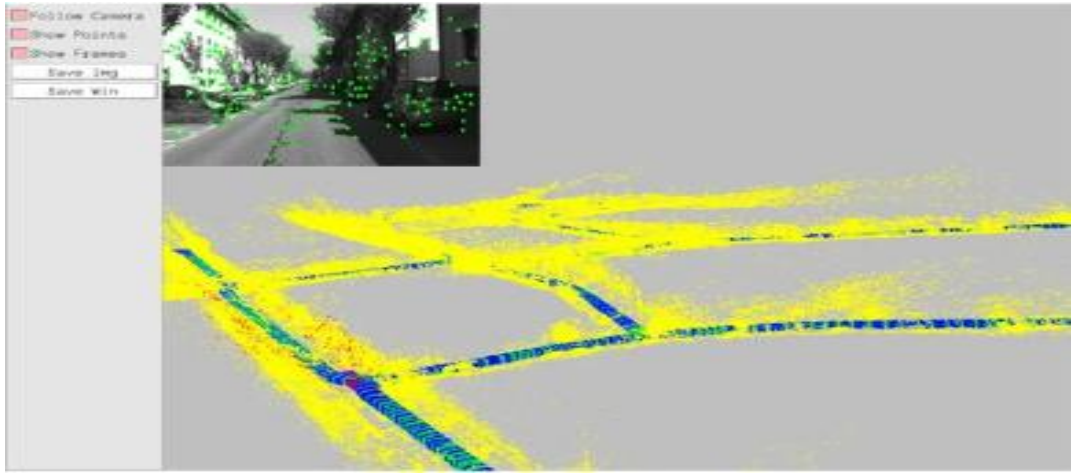
1) Definition

- 자율주행 기술의 일부로 주변 환경 지도를 작성하는 동시에 차량 및 로봇의 위치를 작성된 지도 안에서 추정하는 방법
- SLAM을 통해 생성된 정보를 기반으로 경로 계획 및 장애물 회피 등의 작업을 수행할 수 있음



2) SLAM Pipeline

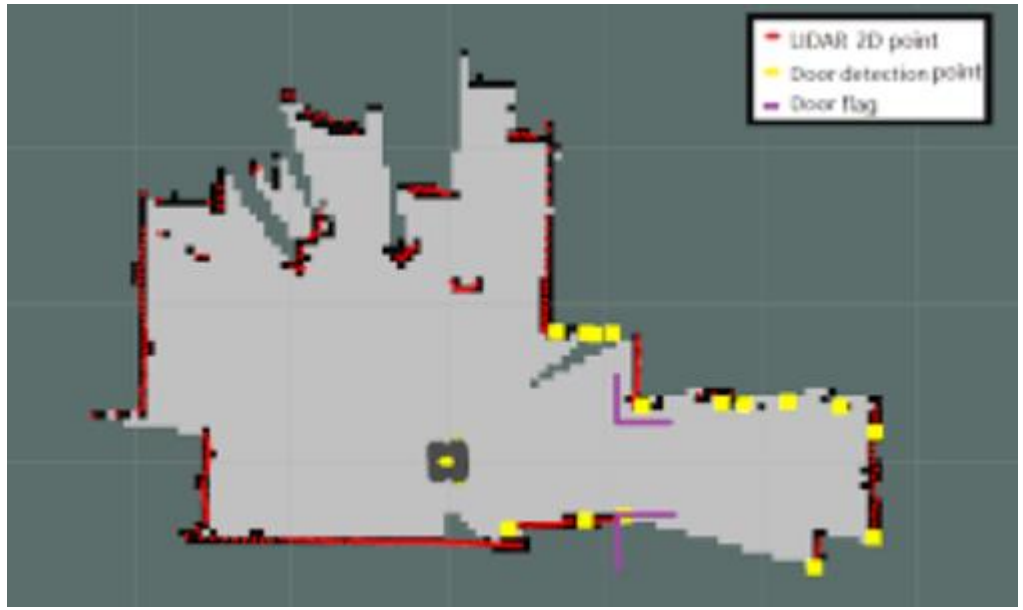
❖ Visual SLAM

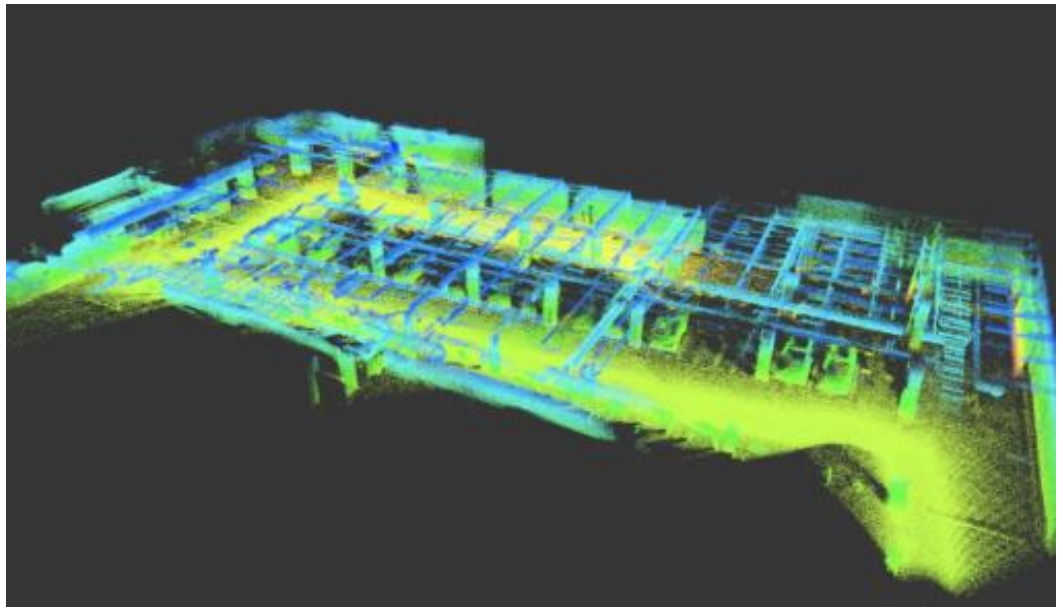


1. Monocular SLAM
   - Depth 정보가 없기에 mono SLAM는 하드웨어 구성은 가장 간단하지만 알고리즘 난이도 높음
2. Stereo SLAM
   - 스테레오 카메라는 고정된 베이스라인(baseline)을 가진 두 개의 카메라를 사용 즉각적인 3D 좌표 취득 가능
3. RGBD SLAM
   - RGBD SLAM은 kinect나 realsense과 같은 센서가 대표적으로 픽셀별 깊이 값을 바로 제공해서 복잡한 기하학적 연산 없이 가능

2) SLAM Pipeline

❖ LiDAR SLAM

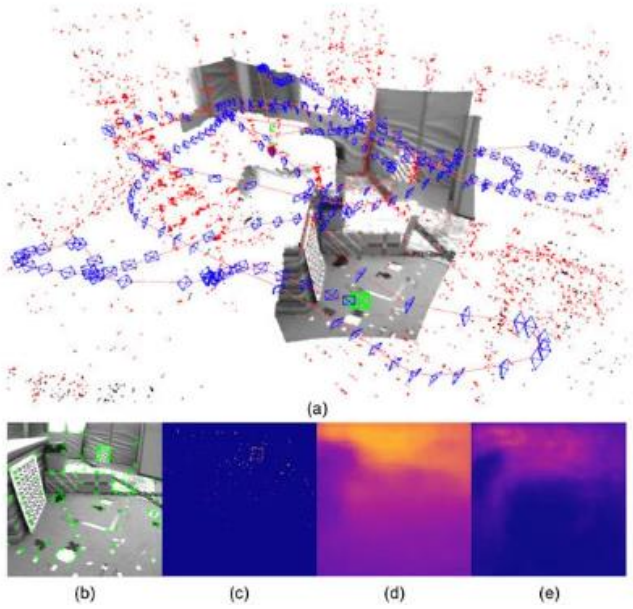


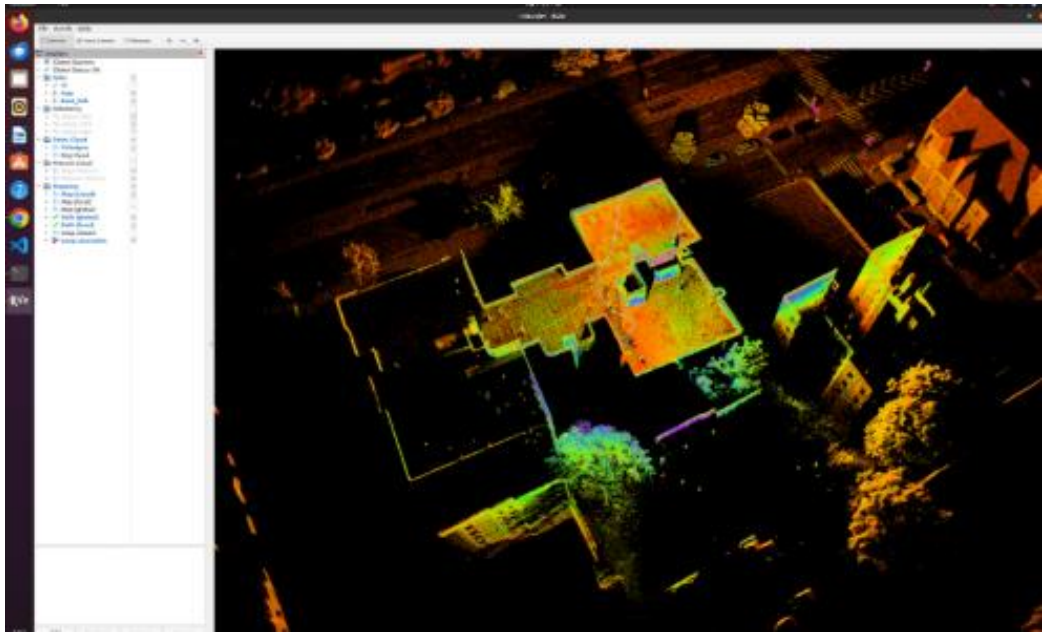


1. 2D Lidar SLAM
   - 2D Lidar는 단일 레이저 라인을 스캔하여 평면 상의 장애물 정보 취득, 실내 로봇에 사용
2. 3D Lidar SLAM
   - 3D Lidar를 통해서 3D 공간의 geometry공간의 정보를 인식

2) SLAM Pipeline

❖ Multi-Modal SLAM





1. Visual Inertial SLAM
   - Camera 및 IMU 센서를 융합 기반
2. Lidar Inertial SLAM
   - LiDAR 및 IMU 센서 융합 기반

Gazebo SLAM Navigation

Gazebo SLAM Navigation

1) Gazebo execution

```
$ cd ~/go2_sim_ws/

$ source install/setup.bash

$ ros2 launch go2_config gazebo_velodyne.launch.py rviz:=true
```

2) Publish TF

```
$ cd ~/go2_sim_ws/

$ source install/setup.bash

$ ros2 run fake_odom odom_tf_broadcaster
```

3) Odom Publisher Node

```
$ cd ~/go2_sim_ws/src/fake_odom/fake_odom/

$ gedit odom_pub.py

$ cd ~/go2_sim_ws/src/fake_odom/

$ gedit setup.py
```

```python
from setuptools import find_packages, setup

package_name = 'fake_odom'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='lab',
    maintainer_email='lab@todo.todo',
    description='TODO: Package description',
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
        'odom_tf_broadcaster = fake_odom.gt_tf:main',
        'odom_local_publisher = fake_odom.odom_pub:main'
        ],
    },
)
```

```python
import rclpy
from rclpy.node import Node

from nav_msgs.msg import Odometry


class LocalOdomPublisher(Node):

    def __init__(self):
        super().__init__('local_odom_publisher')

        self.subscription = self.create_subscription(
            Odometry,
            '/odom/ground_truth',
            self.odom_callback,
            10
        )
        self.publisher = self.create_publisher(Odometry, '/odom/local_ground_truth', 10)

        self.get_logger().info('Odom PublisherStart')

    def odom_callback(self, msg):
        odom_mgs = msg

        # TF frame
        odom_mgs.header.frame_id = 'odom'
        odom_mgs.child_frame_id = 'base_footprint'

        self.publisher.publish(odom_mgs)


def main(args=None):
    rclpy.init(args=args)
    node = LocalOdomPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

3) Odom Publisher Node

```
$ cd ~/go2_sim_ws/

$ colcon build

$ source install setup.bash

$ ros2 run fake_odom odom_local_publisher
```

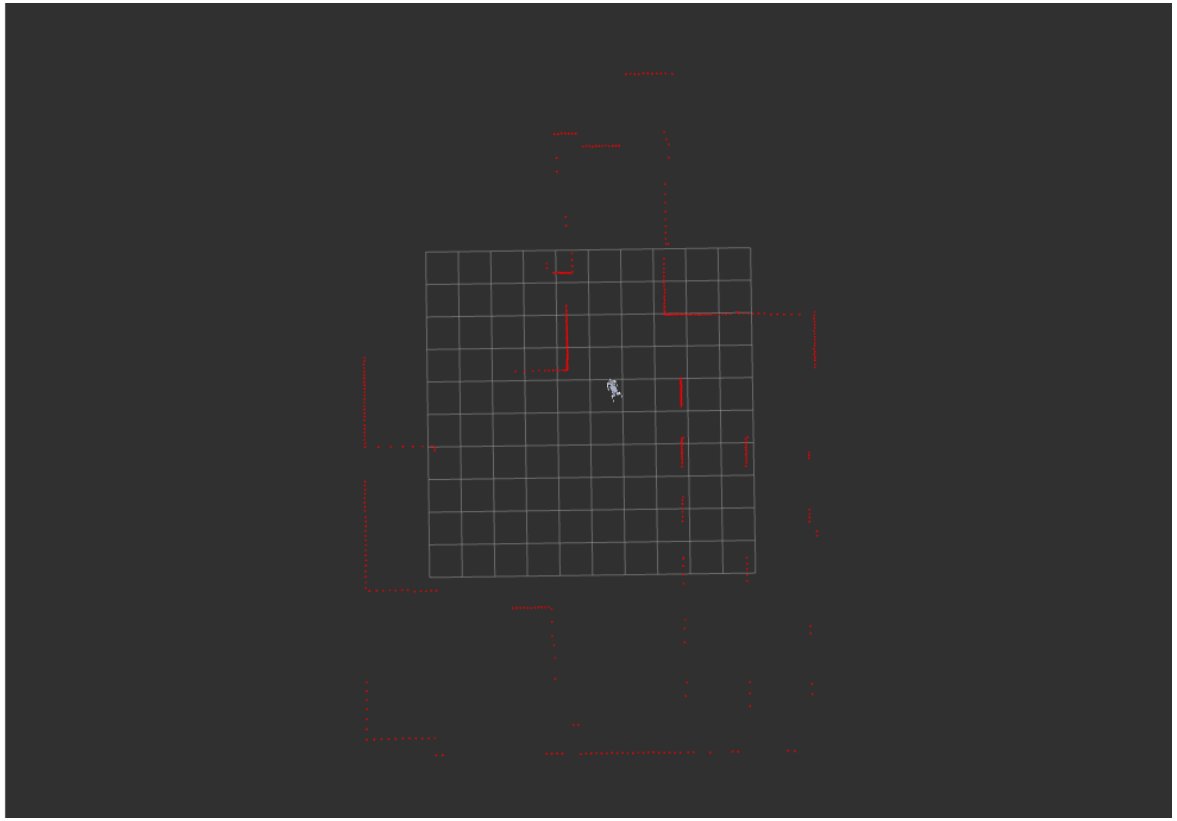
lab@lab: ~/go2_sim_ws

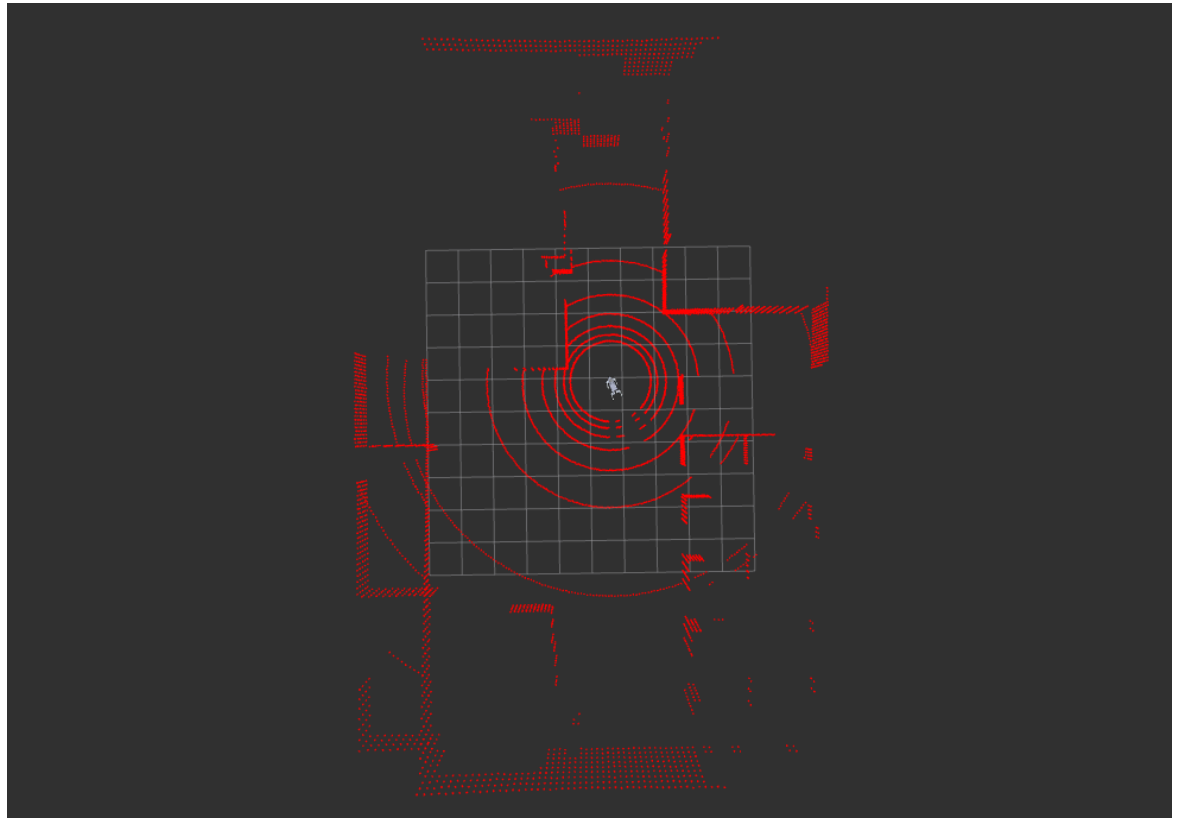
lab@lab: ~/go2_sim_ws | lab@lab: ~/go2_sim_ws | lab@lab: ~/go2_sim_ws | lab@lab: ~/go2_sim_ws

```
/goal_pose [geometry_msgs/msg/PoseStamped]
/hokuyo_frame/scan [sensor_msgs/msg/LaserScan]
/imu/data [sensor_msgs/msg/Imu]
/initialpose [geometry_msgs/msg/PoseWithCovarianceStamped]
/joint_group_effort_controller/controller_state [control_msgs/msg/JointTrajectoryControllerState]
/joint_group_effort_controller/joint_trajectory [trajectory_msgs/msg/JointTrajectory]
/joint_group_effort_controller/state [control_msgs/msg/JointTrajectoryControllerState]
/joint_group_effort_controller/transition_event [lifecycle_msgs/msg/TransitionEvent]
/joint_states [sensor_msgs/msg/JointState]
/joint_states_controller/transition_event [lifecycle_msgs/msg/TransitionEvent]
/odom [nav_msgs/msg/Odometry]
/odom/ground_truth [nav_msgs/msg/Odometry]
/odom/local [nav_msgs/msg/Odometry]
/odom/local_ground_truth [nav_msgs/msg/Odometry]
/odom/raw [nav_msgs/msg/Odometry]
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/performance_metrics [gazebo_msgs/msg/PerformanceMetrics]
/robot_description [std_msgs/msg/String]
/rosout [rcl_interfaces/msg/Log]
/set_pose [geometry_msgs/msg/PoseWithCovarianceStamped]
/tf [tf2_msgs/msg/TFMessage]
/tf_static [tf2_msgs/msg/TFMessage]
/velodyne_points [sensor_msgs/msg/PointCloud2]
lab@lab:~/go2_sim_ws$
```

4) PointCloud(3D) to laserscan(2D)

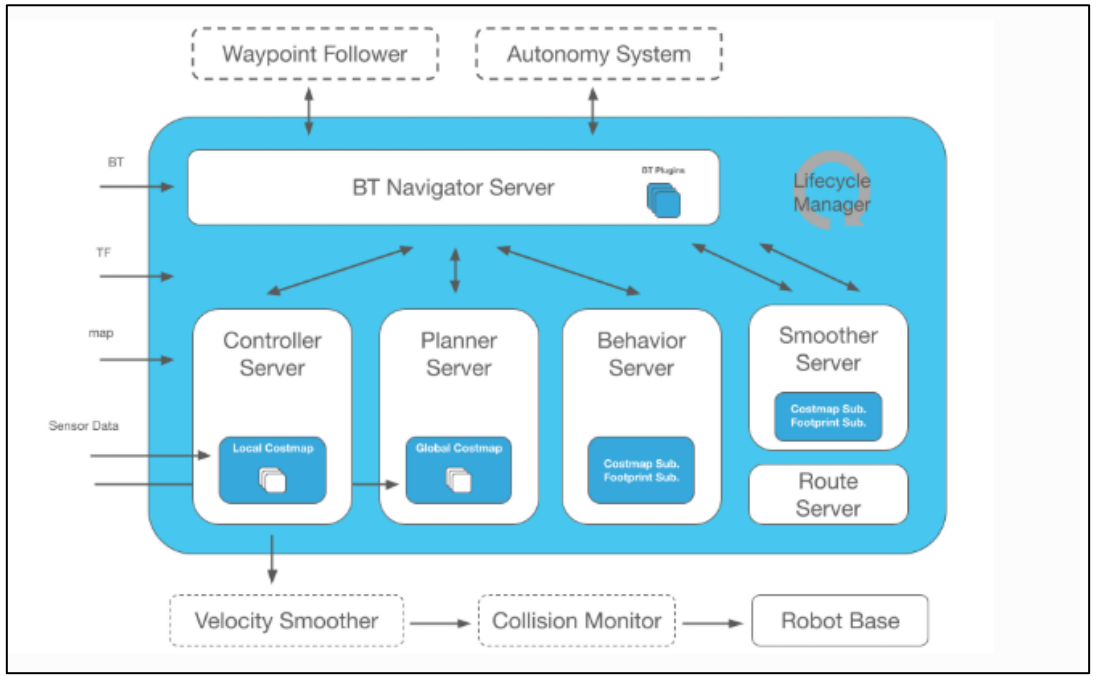
```
$ sudo apt install ros-humble-velodyne-laserscan

$ ros2 run velodyne_laserscan velodyne_laserscan_node  --ros-args  -r cloud:=/velodyne_points
-r scan:=/scan
```





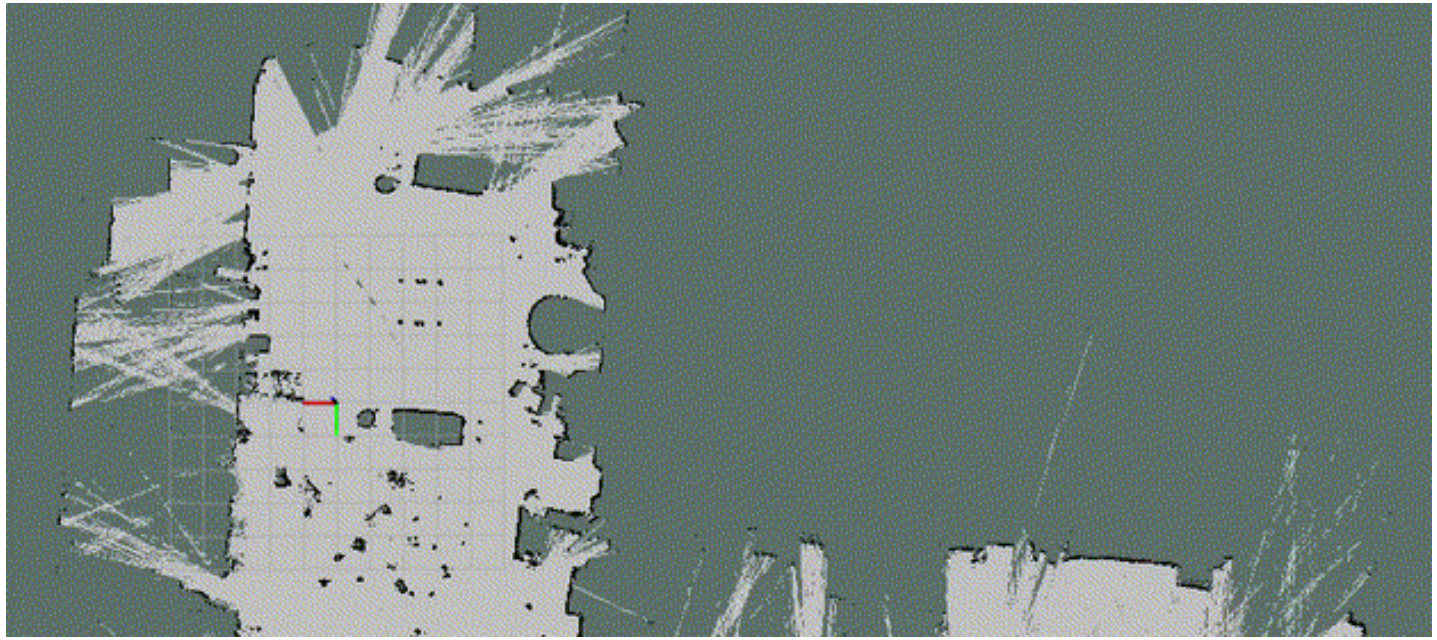
5) Navigation Package Install

```
$ sudo apt install ros-humble-bondcpp

$ sudo apt install libsuitesparse-dev

$ sudo apt install ros-humble-navigation2

$  sudo apt install ros-humble-nav2-bringu
```



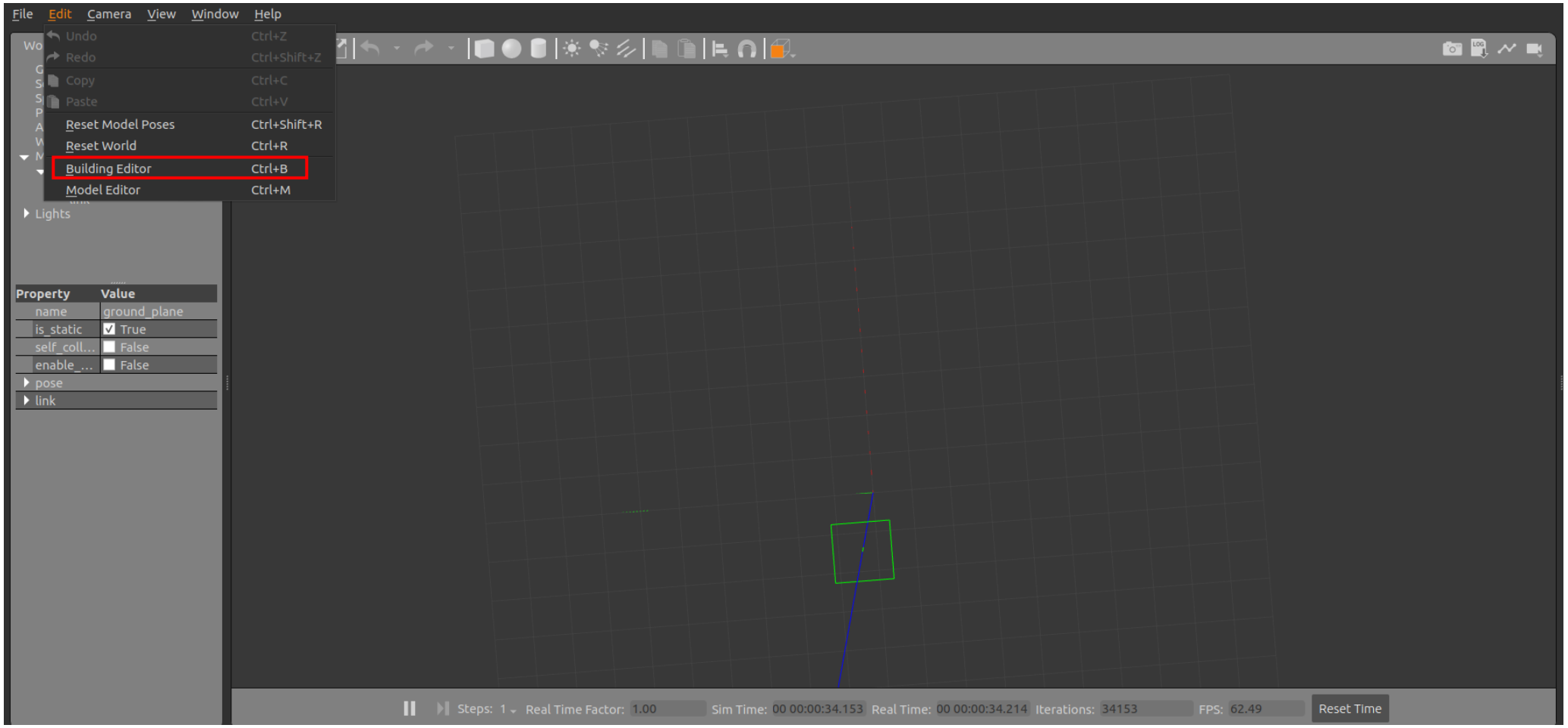
5) SLAM tool Package Install

```
$ cd ~/go2_sim_ws/src/

$ git clone -b humble https://github.com/SteveMacenski/slam_toolbox.git

$ cd ~/go2_sim_ws

$ colcon build
```



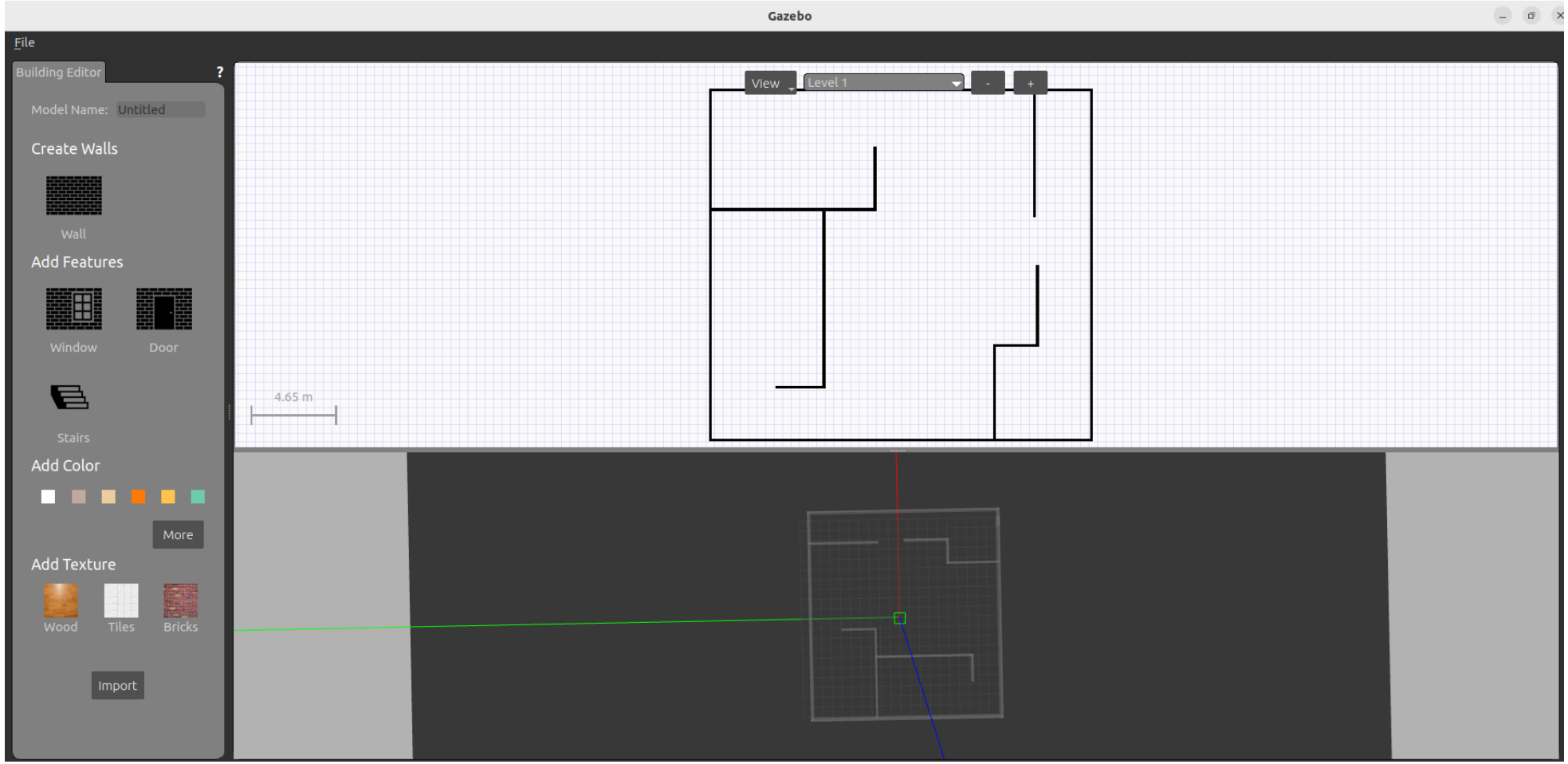
6) Gazebo Map Create

```
$ gazebo
```



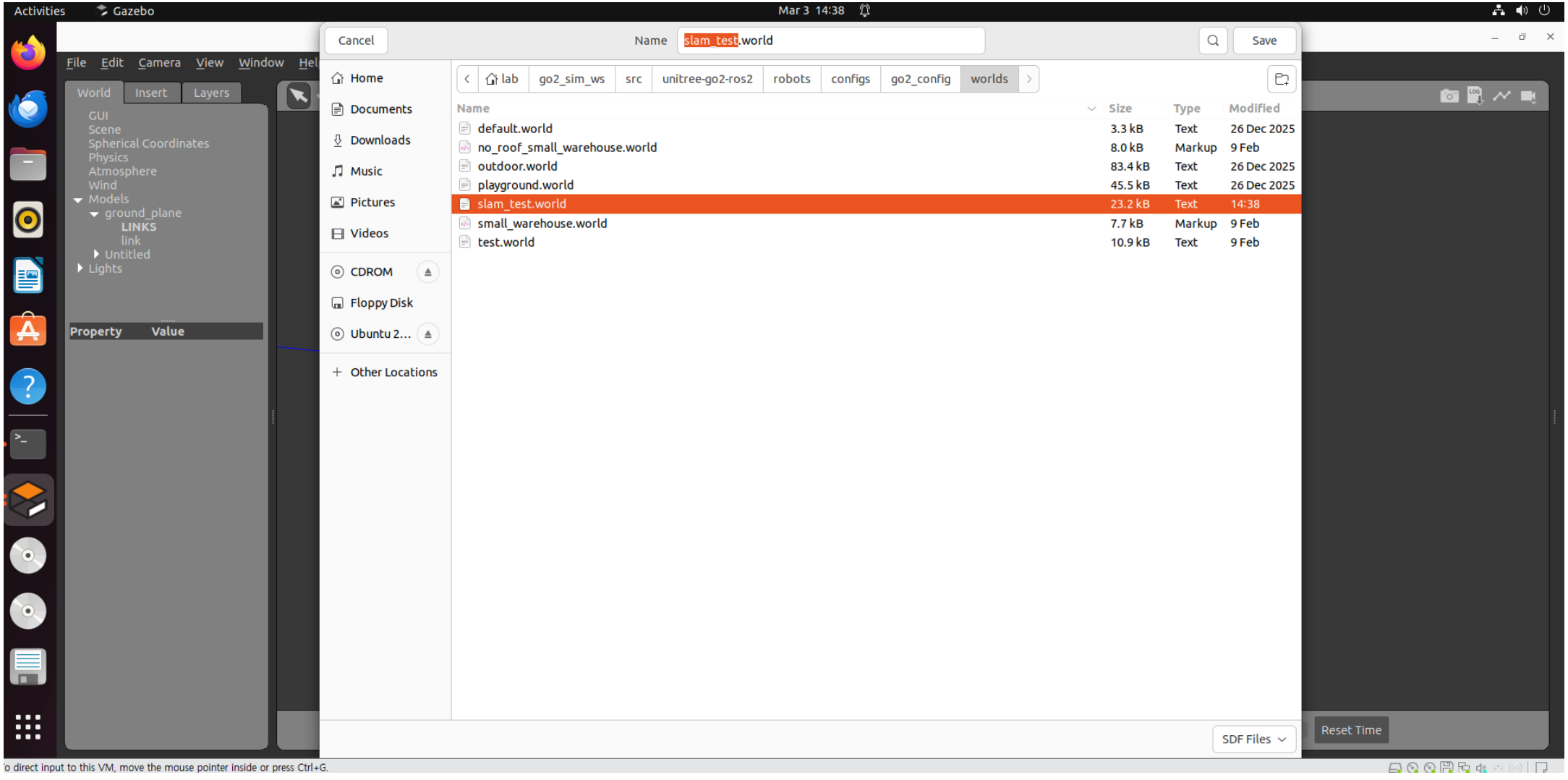
6) Gazebo Map Create

```
$ gazebo
```



6) Gazebo Map Create

```
$ gazebo
```



6) Gazebo Map Create

```
$ cd ~/go2_sim_ws/src/unitree-go2-ros2/robots/configs/go2_config/launch

$ gedit gazebo_velodyne.launch.py

$ cd ~/go2_sim_ws

$ colcon build
```

Open

*gazebo
~/go2_sim_ws/src/unitree-

```python
import os

from launch.actions import SetEnvironmentVariable
import launch_ros
from ament_index_python.packages import get_package_share_directory
from launch_ros.actions import Node

from launch import LaunchDescription
from launch.actions import (
    DeclareLaunchArgument,
    ExecuteProcess,
    IncludeLaunchDescription,
)
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import Command, LaunchConfiguration


def generate_launch_description():

    use_sim_time = LaunchConfiguration("use_sim_time")
    description_path = LaunchConfiguration("description_path")
    base_frame = "base_link"

    config_pkg_share = launch_ros.substitutions.FindPackageShare(
        package="go2_config"
    ).find("go2_config")
    descr_pkg_share = launch_ros.substitutions.FindPackageShare(
        package="go2_description"
    ).find("go2_description")
    joints_config = os.path.join(config_pkg_share, "config/joints/joints.yaml")
    ros_control_config = os.path.join(
        config_pkg_share, "/config/ros_control/ros_control.yaml"
    )
    gait_config = os.path.join(config_pkg_share, "config/gait/gait.yaml")
    links_config = os.path.join(config_pkg_share, "config/links/links.yaml")
    default_model_path = os.path.join(descr_pkg_share, "xacro/robot_VLP.xacro")
    default_world_path = os.path.join(config_pkg_share, "worlds/slam_test.world")
```

7) Gazebo SLAM

Terminal 1

```
$ cd ~/go2_sim_ws/
$ source install/setup.bash
$ ros2 launch go2_config gazebo_velodyne.launch.py rviz:=true
```

Terminal 2

```
$ cd ~/go2_sim_ws/
$ source install/setup.bash
$ ros2 run fake_odom odom_tf_broadcaster
```

Terminal 3

```
$ cd ~/go2_sim_ws/
$ source install/setup.bash
$ ros2 run fake_odom odom_local_publisher
```

7) Gazebo SLAM

Terminal 4

```
$ ros2 run velodyne_laserscan velodyne_laserscan_node  --ros-args  -r cloud:=/velodyne_points
-r scan:=/scan
```

Terminal 5

```
$ cd ~/go2_sim_ws/
$ source install/setup.bash
$ ros2 launch slam_toolbox online_async_launch.py
```

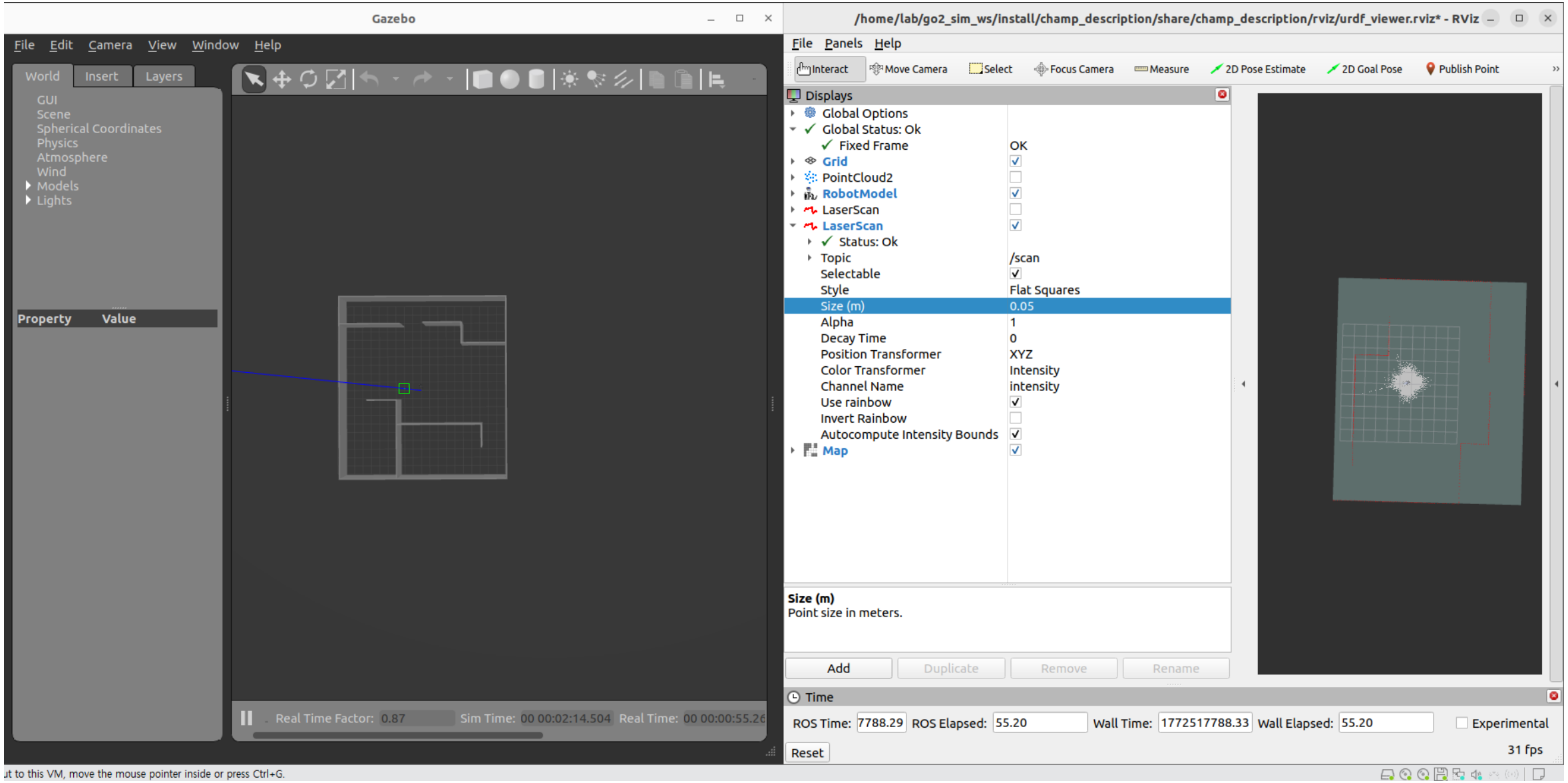
Terminal 6

```
$ ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Terminal 7

```
$ ros2 run nav2_map_server map_saver_cli -f ~/map
```

7) Gazebo SLAM



8) Gazebo Navigation

Terminal 1

```
$ cd ~/go2_sim_ws/
$ source install/setup.bash
$ ros2 launch go2_config gazebo_velodyne.launch.py rviz:=true
```

Terminal 2

```
$ cd ~/go2_sim_ws/
$ source install/setup.bash
$ ros2 run fake_odom odom_tf_broadcaster
```

Terminal 3

```
$ cd ~/go2_sim_ws/
$ source install/setup.bash
$ ros2 run fake_odom odom_local_publisher
```

8) Gazebo Navigation

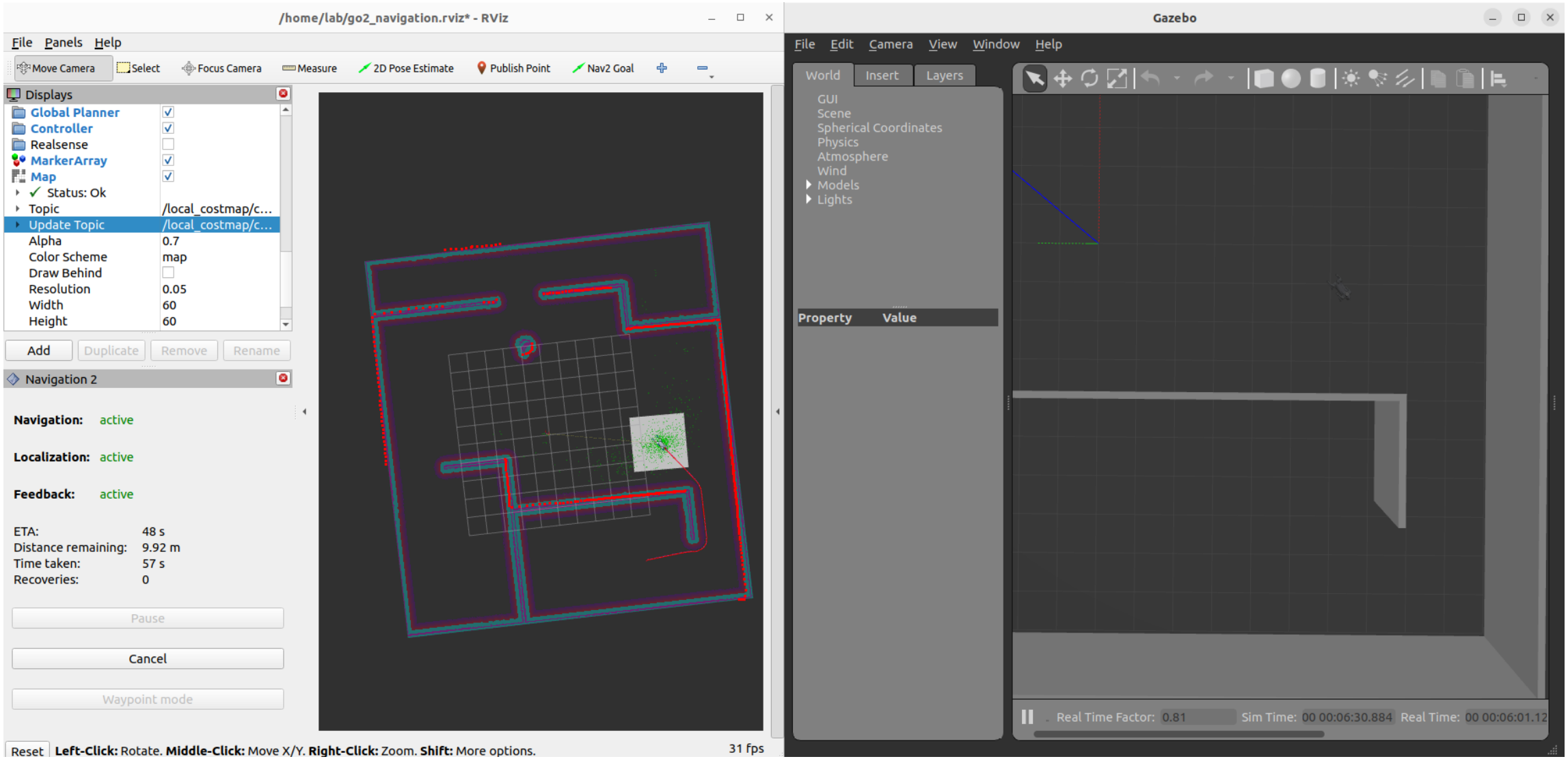
Terminal 4

```
$ ros2 run velodyne_laserscan velodyne_laserscan_node  --ros-args  -r cloud:=/velodyne_points
-r scan:=/scan
```

Terminal 5

```
$ ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Terminal 6

```
$ ros2 launch nav2_bringup bringup_launch.py use_sim_time:=true map:=/home/lab/map.yaml
```

8) Gazebo Navigation